



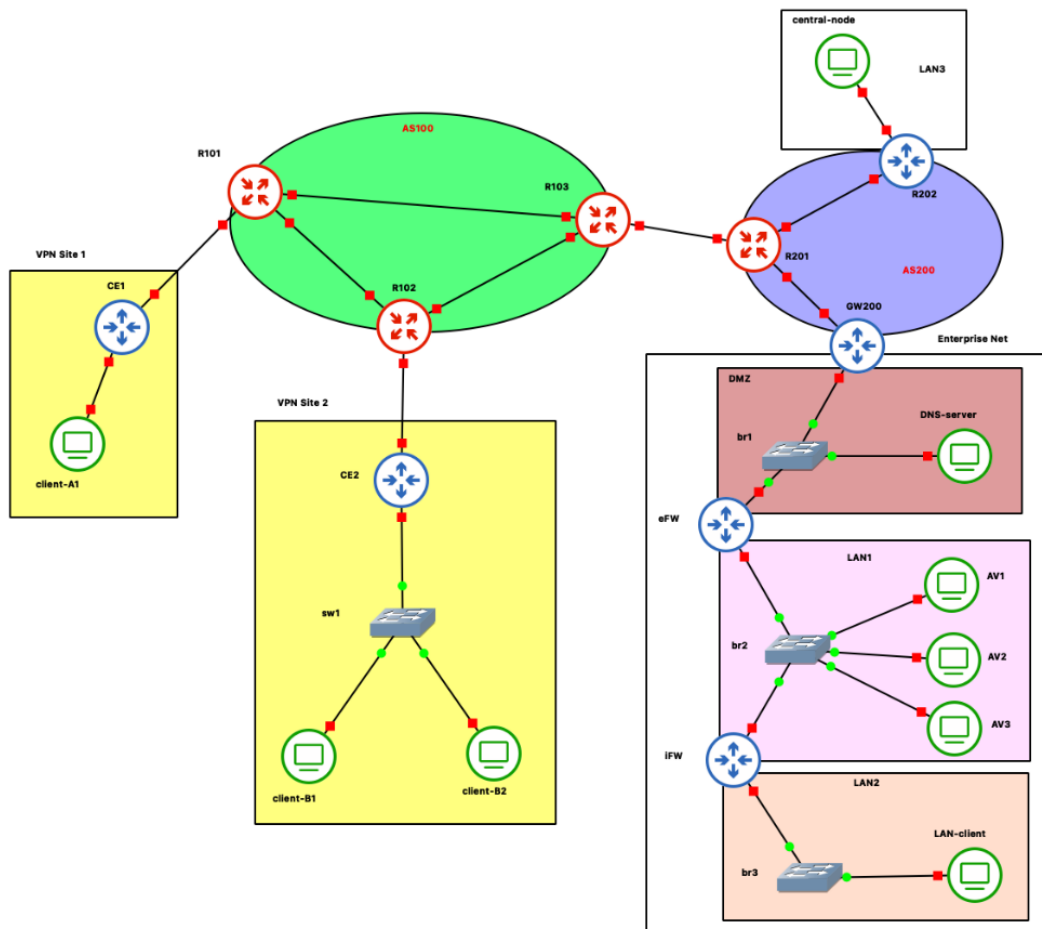
CORSO DI STUDIO IN
Ingegneria Informatica

Report Progetto NSD

Progetto 2

Simone Ercole
Matricola: 0360105

1 Introduzione



Lo scopo del progetto è progettare, configurare e mettere in sicurezza un'infrastruttura di rete complessa, che integra protocolli di routing avanzati, tecnologie di crittografia e politiche di firewalling *stateful*, al fine di supportare sia un ambiente aziendale tradizionale sia una sandbox ad alto isolamento per l'analisi di malware.

L'architettura è strutturata nelle seguenti macro-aree logiche:

- **Core Network (AS100 e AS200):** Il *backbone* dell'infrastruttura si basa sull'interazione di due Autonomous System. L'AS100 funge da dorsale di transito per i siti remoti del cliente, avvalendosi di *OSPF* come protocollo di routing interno e di un'architettura BGP mista (*iBGP* ed *eBGP*) per la propagazione delle rotte. L'AS200 delimita invece il dominio logico della rete *Enterprise*: si interfaccia in *eBGP* con l'AS100 tramite un singolo *Border Router* (R201) e gestisce l'annuncio aggregato del prefisso aziendale (sfruttando rotte di scarto *blackhole*), per poi instradare il traffico legittimo verso il gateway della DMZ.
- **Interconnessione Sicura (VPN e MACsec):** I siti periferici del cliente (VPN Site 1 e VPN Site 2) sono interconnessi tramite un tunnel *IPSec site-to-site*. Inoltre,

all'interno del Sito 2, il traffico di livello datalink è protetto tramite lo standard *MACsec*, garantendo integrità e cifratura trasparente a livello LAN.

- **Rete Enterprise e DMZ:** L'accesso all'infrastruttura aziendale è disciplinato da un gateway (GW200) che controlla gli accessi verso una zona demilitarizzata (DMZ). Al suo interno è ospitato un server DNS autoritativo che eroga servizi Web e garantisce la risoluzione dei nomi autenticata tramite *DNSSEC*.
- **Infrastruttura di Analisi Malware:** Il nucleo operativo e critico del progetto consiste in un ambiente confinato (LAN1) dedicato al testing di binari sospetti. Questa sandbox ospita nodi "Runner" equipaggiati con motori di analisi complementari: scansione a firme (**ClamAV**), analisi comportamentale statica (**Capa**) e *pattern matching* tramite regole custom (**YARA**). L'isolamento è garantito da firewall multipli (eFW e iFW) configurati con *policy* anti-esfiltrazione verso la rete interna (LAN2) o l'esterno. Inoltre, i nodi Runner sono soggetti a *snapshot*, garantendo il ripristino a uno "stato pulito" prima di ogni nuova esecuzione.
- **Automazione Centralizzata:** L'orchestrazione delle scansioni è demandata a uno script Python su un *Central Node* situato in una rete remota (LAN3). Sfruttando un tunnel IPsec dedicato (terminato su eFW e R202) e l'autenticazione SSH a chiavi asimmetriche, il nodo automatizza la distribuzione dei file malevoli (tramite SCP), l'esecuzione parallela delle scansioni e l'aggregazione dei report. Il Central Node funge inoltre da interfaccia NAT (quando necessario) per il download di firme e regole aggiornate, mantenendo i Runner aggiornati.

Nelle sezioni successive del presente documento verranno dettagliate le scelte architetture, le configurazioni dei nodi (es. *Dockerfile* e script di automazione) e le *Security Policies* adottate per soddisfare i requisiti del laboratorio.

2 Autonomous System 100

L'Autonomous System 100 (AS100) funge da dorsale di transito per i siti VPN del cliente e garantisce l'interconnessione globale con l'AS200. L'infrastruttura di routing è implementata sui nodi R101, R102 e R103 avvalendosi della suite *FRRouting*.

2.1 Routing Interno: OSPF

Per garantire la raggiungibilità interna tra i router dell'AS, è stato configurato il protocollo OSPFv2 all'interno dell'Area 0, unica area definita. Ogni router annuncia in OSPF le proprie reti di interconnessione fisiche e, soprattutto, la propria interfaccia virtuale di *Loopback*. L'utilizzo delle interfacce di loopback è cruciale per garantire stabilità alle

successive sessioni iBGP: essendo interfacce logiche, non subiscono variazioni di stato legate alla caduta di un singolo link fisico, prevenendo ricalcoli inutili della topologia.

```
! --- Configurazione OSPF (Estratto R101) ---
router ospf
  ospf router-id 1.1.1.1
  network 1.1.1.1/32 area 0
  network 10.0.0.0/30 area 0
  network 10.0.0.4/30 area 0
exit
```

L'istruzione `network` svolge una duplice funzione: attiva il protocollo sulle interfacce specificate (inviando messaggi Hello per la scoperta dei vicini) e genera gli LSA per informare l'intera Area 0 tramite flooding.

2.2 Routing Esterno: BGP

La distribuzione delle rotte pubbliche e l'interconnessione con l'esterno sono gestite tramite il protocollo BGP, diviso logicamente in due scenari operativi:

iBGP (Internal BGP)

È stata instaurata una topologia *full-mesh* iBGP tra i nodi R101, R102 e R103. Per la creazione e il mantenimento di queste sessioni vengono utilizzati gli indirizzi di loopback propagati precedentemente da OSPF, forzandone l'uso tramite la direttiva `update-source lo`. Inoltre, i router di bordo R101 e R102, e il border router R103, utilizzano la direttiva `next-hop-self` verso i propri peer interni. Questa configurazione è fondamentale in iBGP per risolvere il problema delle rotte esterne: comunica ai vicini interni che per raggiungere una rete esterna il traffico deve essere inviato al router che la sta annunciando, sostituendo il next-hop esterno originale che i router interni non saprebbero come instradare.

```
! --- Configurazione iBGP (Estratto R101) ---
router bgp 100
  bgp router-id 1.1.1.1
  neighbor 2.2.2.2 remote-as 100
  neighbor 2.2.2.2 update-source lo
  neighbor 3.3.3.3 remote-as 100
  neighbor 3.3.3.3 update-source lo
  address-family ipv4 unicast
    neighbor 2.2.2.2 next-hop-self
    neighbor 3.3.3.3 next-hop-self
```

```
exit-address-family
exit
```

eBGP (External BGP)

Il nodo R103 agisce da *Border Router* ed è deputato al peering eBGP verso l'AS200. A differenza dell'ambiente iBGP, in questo scenario il peering è stabilito direttamente sull'indirizzo IP fisico dell'interfaccia condivisa, rendendo superflua la direttiva `update-source`.

```
! --- Configurazione eBGP (Estratto R103) ---
router bgp 100
  bgp router-id 3.3.3.3
  ! Peering eBGP verso AS200 tramite IP fisico
  neighbor 203.0.113.10 remote-as 200
```

2.3 Annuncio delle Rotte Pubbliche

I router R101 e R102 fungono da gateway del provider per i Customer Edge del cliente (Site 1 e Site 2). A tal fine, tramite l'istruzione `network` sotto la *address-family* IPv4, iniettano nel processo BGP i pool di indirizzi pubblici assegnati al cliente:

- **R101** annuncia la rete 203.0.113.0/30, che ospita il gateway CE1.
- **R102** annuncia la rete 203.0.113.4/30, che ospita il gateway CE2.

Questa istruzione ordina al router di iniettare in BGP l'esatta rete specificata, a condizione che questa sia già presente e valida nella tabella di routing locale. Questi prefissi vengono propagati internamente all'AS100 e annunciati esternamente all'AS200, garantendo la raggiungibilità IP bidirezionale, prerequisito per l'instaurazione dei successivi tunnel IPsec.

```
! --- Annuncio Rotte su R102 (Estratto R102) ---
router bgp 100
  address-family ipv4 unicast
    network 203.0.113.4/30
  exit-address-family
```

3 VPN Site 1 e VPN Site 2

Le reti periferiche del cliente sono progettate per garantire l'accesso a Internet ai propri host e, contestualmente, permettere la comunicazione privata e sicura tra le due sedi.

3.1 Configurazione del VPN Site 1

Il nodo CE1 funge da Customer Edge per il Site 1, gestendo la rete LAN interna 10.10.1.0/24. La configurazione di base del gateway e dei client prevede:

- **Routing e Risoluzione Nomi:** CE1 instrada il traffico di default verso il router R101. I client locali (es. client-A1) sono configurati per impostare il CE1 come gateway e interrogare il server DNS autoritativo centralizzato situato nella DMZ aziendale (198.51.100.66).

```
# --- Configurazione Routing e DNS (Estratto CLIENT_A1) ---  
# Imposta il gateway della LAN  
ip route add default via 10.10.1.1  
# Imposta il server DNS  
echo "nameserver 198.51.100.66" > /etc/resolv.conf
```

- **Network Address Translation:** Per consentire ai client di accedere a Internet, su CE1 è configurata una policy di Source NAT (MASQUERADE) tramite `iptables`. Per garantire l'idempotenza dello script a ogni riavvio, la tabella `POSTROUTING` viene preventivamente svuotata (`flush`). È stata posta particolare attenzione nell'implementare una regola di eccezione prioritaria tramite l'azione `ACCEPT`, finalizzata a impedire che il traffico destinato al Site 2 (172.16.20.0/24) subisca la traslazione degli indirizzi prima di essere intercettato dalle policy di IPSec.

```
# --- NAT ed Eccezione VPN (Estratto CE1) ---  
# Svuota le regole precedenti per evitare duplicazioni  
iptables -t nat -F POSTROUTING  
# Eccezione: non nattare il traffico dalla LAN 1 alla LAN 2  
iptables -t nat -A POSTROUTING -s 10.10.1.0/24 -d 172.16.20.0/24 -j ACCEPT  
# Regola generale: natta tutto il resto che esce su Internet  
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

3.2 Configurazione del VPN Site 2 e Sicurezza Datalink (MAC-sec)

Il nodo CE2 gestisce il VPN Site 2 e la relativa LAN 172.16.20.0/24. Questa rete si distingue per l'implementazione di un ulteriore strato di sicurezza logica volto a proteggere il traffico LAN da minacce di Livello 2.

- **Implementazione IEEE 802.1AE (MACsec) e Interfacce Virtuali:** I dispositivi del sito si avvalgono del demone `wpa_supplicant` interfacciato con il modulo kernel `macsec_linux` per autenticarsi reciprocamente. L'interfaccia fisica viene attivata senza IP per fungere da mezzo di trasporto. Il parametro `sleep 10` è essenziale per garantire che il sistema operativo completi l'inizializzazione dell'interfaccia virtuale cifrata (`macsec0`) prima di procedere all'assegnazione dell'indirizzo IP di Livello 3.

```
# --- Attivazione MACsec (Estratto CLIENT_B1) ---
ip link set eth1 up
# Avvia wpa_supplicant con driver macsec_linux in background
wpa_supplicant -i eth1 -D macsec_linux -c /etc/mka.conf -B
# Pausa estesa per consentire la creazione dell'interfaccia
sleep 10
# Assegna l'IP all'interfaccia virtuale cifrata
ip addr add 172.16.20.10/24 dev macsec0
ip link set macsec0 up
```

- **Key Agreement (MKA):** La negoziazione dinamica delle chiavi avviene tramite il protocollo MACsec Key Agreement, basato su una *Pre-Shared Key* definita staticamente nei file di configurazione (`mka.conf`) condivisi tra i nodi.

```
# --- Configurazione MKA (Estratto mka.conf) ---
network={
    key_mgmt=NONE
    macsec_policy=1
    mka_cak=00112233445566778899aabbccddeeff
    mka_ckn=00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff
}
```

Analogamente a CE1, anche CE2 applica policy di NAT in uscita, preservando il traffico destinato al Site 1 tramite apposita *NAT Exemption*.

3.3 Tunnel IPSec Site-to-Site

L'interconnessione privata tra le reti LAN dei due siti è realizzata tramite una Virtual Private Network IPSec, implementata avvalendosi del demone *strongSwan* e configurata tramite `swanctl.conf`.

- **Negoziazione IKEv2:** Il canale di controllo è stabilito tramite il protocollo IKEv2, adottando una chiave pre-condivisa configurata nella sezione **secrets** per l'autenticazione reciproca dei peer (**site1** e **site2**).
- **Suite Crittografica:** Per garantire confidenzialità, integrità e *Perfect Forward Secrecy*, sono state impiegate le seguenti primitive:
 - Canale di controllo (IKE): **aes128-sha256-x25519**.
 - Traffico dati (ESP): **aes128gcm128-x25519**, con rinnovo della chiave dati impostato a 90 minuti.
- **Traffic Selectors:** Le policy IPsec identificano il traffico da sottoporre a cifratura tramite il blocco **children net-net**, vincolando il tunnel esclusivamente ai datagrammi scambiati tra le due sottoreti private (10.10.1.0/24 e 172.16.20.0/24).

```
# --- Configurazione IPsec strongSwan (Estratto CE1/swanctl.conf) ---
connections {
    site1-site2 {
        version = 2
        proposals = aes128-sha256-x25519

        local { auth = psk, id = site1 }
        remote { auth = psk, id = site2 }

        children {
            net-net {
                # Traffic Selectors
                local_ts = 10.10.1.0/24
                remote_ts = 172.16.20.0/24

                rekey_time = 5400
                esp_proposals = aes128gcm128-x25519
            }
        }
    }
}
```


4 Autonomous System 200 e Perimetro Enterprise

Il dominio di routing dell'infrastruttura aziendale è governato dall'Autonomous System 200, il quale funge da perimetro logico per l'esposizione controllata dei servizi (DMZ) e fornisce connettività aggregata alle reti interne (LAN3 e Rete Enterprise). L'architettura si articola sui nodi R201, GW200 e R202.

4.1 Routing di Frontiera (R201)

Il router R201 costituisce l'unico *BGP speaker* dell'AS200, incaricato al peering eBGP con il nodo R103 dell'AS100. Per ottimizzare l'aggregazione degli annunci e garantire stabilità alla *routing table* globale, R201 implementa una rotta statica di *discard* (**blackhole**) per l'intero macro-prefisso aziendale 198.51.100.0/24. Questa architettura assicura che il demone BGP annunci all'esterno la classe /24 in modo ininterrotto, indipendentemente dallo stato dei link interni. La corretta consegna dei datagrammi è poi delegata all'algoritmo di *Longest Prefix Match*: ad esempio, i pacchetti legittimi diretti alla DMZ (198.51.100.64/26) vengono instradati verso GW200, mentre il traffico indirizzato a porzioni non allocate del prefisso collassa nella rotta di scarto.

```
! --- Configurazione Routing e BGP (Estratto R201) ---
! Rotta di "pull-up" (blackhole) per far annunciare la /24 al BGP
ip route 198.51.100.0/24 blackhole
! Indirizzamento specifico per la DMZ verso GW200 (Longest Prefix Match)
ip route 198.51.100.64/26 198.51.100.6

router bgp 200
  address-family ipv4 unicast
    ! Annuncio dell'intera /24 all'esterno
    network 198.51.100.0/24
  exit-address-family
```

4.2 Gateway Enterprise (GW200)

Il nodo GW200 non partecipa attivamente allo scambio BGP, delegando l'instradamento in uscita a una rotta di default verso R201. Il suo ruolo strategico è quello di *edge router* e firewall perimetrale.

- **Routing Interno:** GW200 funge da punto di snodo, instradando il traffico destinato alle reti private interne (LAN1 e LAN2) verso il firewall di secondo livello, *eFW* (198.51.100.67).

- **Security Policy (Firewall):** Il filtraggio dei pacchetti è implementato tramite `iptables`. Il firewall adotta una policy *Deny-by-Default* (DROP sulla catena FORWARD), autorizzando esplicitamente il solo traffico strettamente necessario.
- **Servizi Esposti e Connection Tracking:** Le regole in ingresso abilitano l'accesso ai servizi pubblici della DMZ (DNS su UDP 53 e HTTP su TCP 80) e il transito del traffico IPsec (IKE su UDP 500/4500, e protocollo ESP) diretto verso *eFW*. La bidirezionalità delle comunicazioni originate dall'interno è garantita dal modulo di *connection tracking* (ESTABLISHED, RELATED).

```
# --- Configurazione Firewall (Estratto GW200) ---
# Deny-by-default e Connection Tracking
iptables -P FORWARD DROP
iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT

# Esposizione Servizi DMZ (DNS e HTTP)
iptables -A FORWARD -i eth0 -d 198.51.100.66 -p udp --dport 53 -j ACCEPT
iptables -A FORWARD -i eth0 -d 198.51.100.66 -p tcp --dport 80 -j ACCEPT

# Transito VPN verso eFW (.67)
iptables -A FORWARD -i eth0 -d 198.51.100.67 -p udp --dport 500 -j ACCEPT
iptables -A FORWARD -i eth0 -d 198.51.100.67 -p udp --dport 4500 -j ACCEPT
iptables -A FORWARD -i eth0 -d 198.51.100.67 -p esp -j ACCEPT
```

4.3 R202 - LAN3 Gateway

Il nodo R202 è il router edge dedicato alla LAN3 (10.30.1.0/24), rete che ospita il *Central Node* di orchestrazione. Anch'esso privo di configurazioni BGP dinamiche, si affida a R201 per il transito verso l'esterno. R202 agisce come endpoint di un tunnel IPsec IKEv2 dedicato, negoziato direttamente con il firewall *eFW*. Questa architettura è vitale per la sicurezza della sandbox malware, poiché confina il traffico di distribuzione dei binari infetti e la reportistica (tramite SSH/SCP) all'interno di un perimetro crittografato, isolandolo dal normale traffico aziendale.

```
# --- Configurazione Tunnel Sandbox (Estratto R202/swanctl.conf) ---
connections {
    lan3-lan1 {
        local_addrs = 198.51.100.2
        remote_addrs = 198.51.100.67

        local { auth = psk, id = r202 }
```

```
remote { auth = psk, id = efw }

children {
    av-tunnel {
        local_ts = 10.30.1.0/24
        remote_ts = 10.20.1.0/24
        esp_proposals = aes128gcm128-x25519
    }
}
}
```

Per garantire che il traffico di orchestrazione generato dal Central Node raggiunga i Runner senza essere bloccato, è stata configurata un'eccezione NAT su R202. Questa regola impedisce che i datagrammi diretti alla LAN1 (10.20.1.0/24) vengano "mascherati" con l'IP pubblico del router prima di essere elaborati dal demone strongSwan.

```
# --- NAT ed Eccezione Management (Estratto R202) ---
iptables -t nat -F POSTROUTING
iptables -t nat -A POSTROUTING -s 10.30.1.0/24 -d 10.20.1.0/24 -j ACCEPT
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

5 Rete Enterprise

La Rete Enterprise è strutturata su più livelli per garantire maggiore sicurezza. L'infrastruttura è stata divisa in zone distinte, per separare: i servizi accessibili da Internet, le postazioni di lavoro interne e l'area isolata in cui vengono analizzati i malware. Questa separazione è assicurata dall'impiego di due firewall distinti (eFW e iFW).

5.1 Demilitarized Zone (DMZ) e Servizi Esposti

La DMZ ospita le risorse aziendali accessibili dall'esterno, confinate in un perimetro di rete dedicato. Al suo interno è posizionato un server che funge da Web Server e DNS Autoritativo per il dominio `nsdcourse.xyz`.

- **Sicurezza DNS e File di Configurazione:** Il servizio di risoluzione nomi, basato su `bind9`, è strutturato seguendo un approccio modulare. Il file `named.conf.options` è configurato esplicitamente per non accettare query ricorsive (`recursion no;`), autorizzando pubblicamente le sole richieste autoritative (`allow-query { any; }`).

```
# --- Configurazione DNS (Estratti da named.conf e db.nsdcourse.xyz) ---
# named.conf.options (Comportamento globale)
options {
    recursion no;
    allow-query { any; };
    dnssec-validation auto;
};
```

- **Dichiarazione di Zona e Record:** Il file `named.conf.local` definisce la zona *master* per `nsdcourse.xyz`, agganciando direttamente il file di zona post-firma crittografica (`db.nsdcourse.xyz.signed`). Il file testuale dei record (`db.nsdcourse.xyz`) contiene le mappature essenziali: oltre ai parametri di validità SOA, definisce i record A che instradano i servizi `ns1` e `www` sull'indirizzo IP pubblico del DNS-Server (198.51.100.66).

```
# named.conf.local (Dichiarazione zona)
zone "nsdcourse.xyz" {
    type master;
    file "/etc/bind/db.nsdcourse.xyz.signed";
};

# db.nsdcourse.xyz (Estratto record di zona)
ns1      IN      A      198.51.100.66
www      IN      A      198.51.100.66
```

- **Firma Crittografica (DNSSEC):** Per prevenire attacchi di manipolazione (es. *DNS Cache Poisoning*), la zona è protetta generando chiavi KSK e ZSK. Lo script di inizializzazione firma i record della zona originaria implementando il meccanismo NSEC3 (-3), avvalendosi del generatore `/dev/urandom` per creare un *salt* monouso.

```
# --- Generazione Chiavi e Firma DNSSEC (Estratto setup_services.sh) ---
dnssec-keygen -a RSASHA256 -b 2048 -n ZONE nsdcourse.xyz
dnssec-keygen -a RSASHA256 -b 4096 -n ZONE -f KSK nsdcourse.xyz

# Firma della zona con salt dinamico (NSEC3)
dnssec-signzone -A -3 $(head -c 1000 /dev/urandom | shasum | cut -b 1-16) \
    -N INCREMENT -o nsdcourse.xyz -t db.nsdcourse.xyz
```

- **Web Server:** Sulla medesima istanza è in ascolto un server web Apache2, configurato tramite un *VirtualHost* dedicato sulla porta TCP 80, il quale espone il sito `www.nsdcourse.xyz` e ne mappa la *DocumentRoot* in modalità di sola lettura.

```
# --- Configurazione VirtualHost Apache2 (Estratto setup_services.sh) ---
<VirtualHost *:80>
    ServerName www.nsdcourse.xyz
    ServerAlias ns1.nsdcourse.xyz
    DocumentRoot /var/www/nsdcourse
    <Directory /var/www/nsdcourse>
        Options -Indexes +FollowSymLinks
        AllowOverride None
        Require all granted
    </Directory>
    ErrorLog ${APACHE_LOG_DIR}/nsdcourse_error.log
    CustomLog ${APACHE_LOG_DIR}/nsdcourse_access.log combined
</VirtualHost>
```

5.2 External Firewall (eFW)

Il nodo eFW agisce da snodo centrale per la separazione delle reti. È posizionato tra la DMZ, l'ambiente di analisi (LAN1) e la rete aziendale (LAN2).

- **Gestione VPN:** eFW funziona da terminatore per il tunnel IPsec instaurato con R202, creando un canale di comunicazione crittografato tra il Central Node e i nodi Runner.
- **Isolamento della LAN1:** Per evitare la fuoriuscita di dati o la diffusione dei malware analizzati, eFW applica una politica restrittiva tramite `iptables`. Blocca tutto il traffico in uscita dalla LAN1, permettendo come unica eccezione il traffico diretto al Central Node all'interno del tunnel VPN.
- **Accesso a Internet per la LAN2:** Il firewall permette ai client della rete aziendale di navigare su Internet utilizzando la traslazione degli indirizzi, nascondendo così gli indirizzi IP privati all'esterno.

```
# --- Isolamento Isolamento LAN1 e NAT LAN2 (Estratto eFW/start.sh) ---
iptables -P FORWARD DROP
```

```
# Autorizza le comunicazioni LAN1 <-> LAN3 (Tunnel VPN per analisi malware)
```

```
iptables -A FORWARD -i eth1 -s 10.20.1.0/24 -d 10.30.1.0/24 -j ACCEPT
iptables -A FORWARD -s 10.30.1.0/24 -d 10.20.1.0/24 -j ACCEPT
```

```
# Blocca esplicitamente qualsiasi altro traffico in uscita dai Runner
iptables -A FORWARD -i eth1 -s 10.20.1.0/24 -j DROP
```

```
# NAT (Masquerade) per permettere la navigazione della LAN2
iptables -t nat -A POSTROUTING -s 10.20.2.0/24 -o eth0 -j MASQUERADE
```

5.3 Internal Firewall (iFW) e LAN2

Il firewall interno iFW fornisce l'ultimo livello di protezione, con lo scopo primario di difendere la rete aziendale (LAN2) da eventuali minacce provenienti dall'area di analisi (LAN1). Il firewall è configurato con una policy predefinita di blocco totale (DROP). Tuttavia, avvalendosi del tracciamento dello stato delle connessioni, permette ai client aziendali di iniziare connessioni verso l'esterno (ad esempio per interrogare il server DNS in DMZ) e di riceverne le relative risposte. Al contrario, qualsiasi richiesta di nuova connessione diretta dall'esterno verso la LAN2 viene scartata.

```
# --- Sicurezza Stateful (Estratto iFW/start.sh) ---
iptables -P FORWARD DROP
# Traffico in uscita: Autorizzato
iptables -A FORWARD -i eth1 -o eth0 -s 10.20.2.0/24 -j ACCEPT
# Traffico in entrata: Solo risposte a connessioni legittime
iptables -A FORWARD -i eth0 -o eth1 -m state --state ESTABLISHED,RELATED -j ACCEPT
```

6 Infrastruttura di Analisi Malware

Il nucleo operativo a più alta criticità del progetto è rappresentato da una *sandbox* automatizzata per l'analisi logica di eseguibili potenzialmente malevoli. L'architettura adotta un paradigma distribuito, separando la fase di orchestrazione (Central Node) dagli ambienti di esecuzione isolati (Runner).

6.1 Gestione Automatizzata delle Credenziali e Sicurezza SSH

Trattandosi di un ambiente di esecuzione per codice ostile, l'intero ecosistema è progettato con una logica di automazione per il provisioning delle credenziali.

- **Provisioning del Central Node:** All'avvio, il nodo di orchestrazione non dispone fisicamente delle chiavi in chiaro. Per garantire persistenza e coerenza nell'accesso, la coppia di chiavi crittografiche viene generata una singola volta, in fase di

setup iniziale. Ad ogni successivo avvio del nodo, la chiave privata RSA definitiva viene iniettata dinamicamente nel sistema. Contestualmente, il demone SSH viene istruito per bypassare il controllo interattivo dell'impronta crittografica degli host (`StrictHostKeyChecking no`), permettendo agli script Python di connettersi ai container senza interruzioni e blocchi prompt.

```
# --- Provisioning Dinamico delle Chiavi (Estratto LAN3-CentralNode/start.sh)
mkdir -p /root/.ssh
chmod 700 /root/.ssh
# Decodifica la chiave privata da Base64 per evitare errori di formattazione
echo "LS0tLS1CRUdJTiBP...[TRUNCATED]...==" | base64 -d > /root/.ssh/id_rsa
chmod 600 /root/.ssh/id_rsa
# Disabilita il prompt di verifica dell'host per l'automazione totale
echo "StrictHostKeyChecking no" > /root/.ssh/config
```

- **Autenticazione Asimmetrica sui Runner:** I nodi antivirus (ClamAV, Capa, YARA) blindano i propri accessi disabilitando l'autenticazione basata su password. I loro script di avvio (`start.sh`) creano dinamicamente la directory `.ssh` con permessi restrittivi (`chmod 700`) e iniettano la chiave pubblica nel file `authorized keys`. Questo garantisce che solo il Central Node, detentore della chiave privata associata, possa accedere ed eseguire i payload.

```
# --- Iniezione Chiave Pubblica (Estratto start.sh dei Runner) ---
mkdir -p /root/.ssh
chmod 700 /root/.ssh

# Iniezione della chiave pubblica autorizzata per il Central Node
echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIIJNhRugpzhST8NV/tw9+aPcFMMqQQheTUr"
chmod 600 /root/.ssh/authorized_keys

# Riavvio del demone per applicare le restrizioni
service ssh restart
```

6.2 I Nodi Runner (LAN1)

La LAN1 ospita tre ambienti virtualizzati separati tramite container, realizzati avvalendosi di tre specifici Dockerfile (`Dockerfile.clamav`, `Dockerfile.capa`, `Dockerfile.yara`), ciascuno equipaggiato con un motore di analisi complementare:

- **ClamAV:** incaricato del rilevamento convenzionale basato su firme. L'analisi viene condotta disabilitando il riepilogo a schermo, per facilitare l'ingestione dei log da parte del nodo centrale.
- **Capa:** integrato per l'analisi statica comportamentale e l'estrazione delle capacità operative del malware. All'interno del proprio `Dockerfile`, il nodo è istruito per scaricare direttamente online (tramite `wget`) le regole ufficiali aggiornate dal repository di Mandiant (`mandiant/capa-rules`) durante la fase di build. L'interrogazione tramite lo script `scan.sh` invoca poi esplicitamente il motore, passandogli come parametro proprio la cartella delle regole appena scaricata.
- **YARA:** impiegato per il *pattern matching* avanzato, testato in questo scenario tramite regole customizzate (come il pattern di validazione EICAR) scritte e iniettate direttamente nel filesystem durante la creazione dell'immagine. Tali regole sono state strutturate in modo da rimanere comunque aggiornabili ed espandibili dinamicamente anche in fasi successive, senza la necessità di dover ricreare il container.

6.3 Il Central Node (LAN3) e Orchestrazione

Il *Central Node* funge da "cervello" dell'architettura. L'orchestrazione è garantita da uno script nativo in Python (`distribute_malware.py`) che automatizza l'intero ciclo di vita dell'analisi:

1. **Distribuzione Sicura:** Il Central Node itera sugli indirizzi IP dei Runner; per ognuno, invia il file sospetto avvalendosi del protocollo `scp` associato alla chiave privata decodificata in precedenza.
2. **Esecuzione Automatizzata:** Successivamente alla copia, lo script innesca l'analisi tramite una sessione SSH remota, richiamando lo script `scan.sh` del nodo e passandogli il percorso del file malevolo.
3. **Aggregazione Telemetrica:** Lo standard output generato dai nodi viene catturato e formattato a terminale, fornendo un output unificato.

```
# --- Orchestrazione Malware (Estratto distribute_malware.py) ---
file_name = os.path.basename(target_file)
remote_path = f"/tmp/{file_name}"

for ip in RUNNERS:
    # Trasferimento del payload tramite SCP autenticato a chiave
    subprocess.run(["scp", "-i", SSH_KEY, target_file,
```



```
f"root@{ip}:{remote_path}"], check=True)

# Esecuzione remota (Agentless) e cattura dell'output
res = subprocess.run(["ssh", "-i", SSH_KEY, f"root@{ip}",
                     f"/root/scan.sh {remote_path}"],
                     capture_output=True, text=True)

print(f"[+] Report: {res.stdout.strip()}")
```